PRACTICE WORKSHOP

Workshop 3: Encapsulation

In this workshop you design classes that protect their internal state, exposing only what callers need through well-defined methods.

Learning Objectives

- Declare fields as private and expose controlled access through getters/setters.
- Validate input inside setters instead of trusting the caller.
- Write and use constructors, including overloaded constructors.
- Explain, in your own words, why encapsulation makes code easier to maintain.

Exercises

Exercise 1: BankAccount Class

Design a BankAccount class with a private balance field. Provide deposit(double amount) and withdraw(double amount) methods that reject negative amounts and withdrawals that would overdraw the account, printing an appropriate message instead.

Hints:

- Never let balance become a public field.
- Consider what withdraw should do when amount > balance.

```
public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        // TODO
    }

    public void deposit(double amount) {
        // TODO
    }

    public boolean withdraw(double amount) {
        // TODO: return true if successful, false otherwise
    }
}
```

Exercise 2: Student Record

Create a Student class with private fields for name and score (0-100). The setScore method should clamp any out-of-range value into the valid range rather than accepting it as-is. Add a getLetterGrade() method that derives a letter grade from the score.

Hints:

- Clamping means: if score > 100, store 100; if score < 0, store 0.

Exercise 3: Rectangle with Overloaded Constructors

Write a Rectangle class with private width and height fields. Provide one constructor that takes both dimensions and another that takes a single value to build a square. Add getArea() and getPerimeter() methods.

Hints:

- The single-argument constructor can call the two-argument one with this(side, side).

Stretch Goal (optional)

Add an equals(Object other) override to Rectangle that returns true when two rectangles have the same area, and explain why comparing by area (rather than by width/height) might be a questionable design choice.